AOSP 进阶丛书 · 第十册 · ANDROID 13 (API 33)

网络栈源码全解

Connectivity 模块化新布局、网络评分与验证四探针、netd 与 eBPF 流量记账、DNS 与 Private DNS 三态、VPN tun 机制与 Tethering offload——一次请求要过的全部系统关卡。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：网络是一组"被治理的路径" 2
第一章 全链路：一次请求要过几道关 2
第二章 网络请求与评分：怎么选默认网 3
第三章 验证：这网能不能信 4
第四章 netd 与 eBPF：流量从记账到限速 5
第五章 DNS：Private DNS 三态 7
第六章 VPN：tun 设备与路由劫持 7
第七章 设计决策八问 8
附录 A 观测实操 9
附录 B 快问快答（16 条） 9
附录 C 自检题 10
第八章 ConnectivityService 决策内核：注册与切网全链 10
第九章 NetworkMonitor 评估状态机全解 13
第十章 eBPF 数据面：cgroup 计账程序与热点卸载 18
第十一章 DNS 解析内核：dnsproxyd 协议与 DoT 校验 21
教科书习题集（分级） 24
术语表 27
排障手册：网络栈问题定位 28
网络栈核心速查表 30

本册速览

模块化新布局：Connectivity→packages/modules/Connectivity（Mainline）
评分与验证：NetworkRanker 挑默认网 · NetworkMonitor 四探针
netd 与 eBPF：tagSocket→uid_tag_map→流量账本 · Tethering offload
DNS 三态与 VPN：Private DNS(DoT) · tun 设备与 protect 防环
设计决策八问 + 附录 A 观测表

前言：网络是一组"被治理的路径"

对应用开发者网络是"默认有网"；对系统开发者网络是**一组被策略治理的路径**：哪个网络可用（验证）、谁优先（评分）、谁的流量走哪（路由）、流量记到谁头上（eBPF 记账）、DNS 是否加密（Private DNS）、全量代理怎么不绕圈（VPN protect）。本册沿这六问展开。

路径新布局（读码第一坑）：本树（Android 13）里 Connectivity 已 Mainline 化——`packages/modules/Connectivity/`（记 CS=…/service/src/com/android/server）；NetworkMonitor 在**独立的 NetworkStack 模块**（packages/modules/NetworkStack/）；netd 在 system/netd/server/（main.cpp:111）；而 Vpn.java 仍在 frameworks/base/services/core/…/connectivity/（:179）。旧教程的 frameworks/base/services/.../connectivity 路径在本树**只余 Vpn.java 一个活化石**。

第一章 全链路：一次请求要过几道关

设计洞察 · 学习目标

1. 说出 CS/NetworkMonitor/netd 三段的分工与各自主责；2. 解释本树为何要去 packages/modules/Connectivity 找主代码；3. 用 dumpsys connectivity 找到一个网络的验证状态与分数；4. 画出一次 HTTPS 请求经过的四大关卡



图 1 网络栈全链路：连接管理（CS）→验证（NetworkMonitor）→数据落地（netd/eBPF）三段，含模块化迁移对照表

三段分工一句话：**CS 管决策（谁当默认网）、NetworkMonitor 管验证（这网能不能信）、netd 管落地（DNS/规则/记账）**。App 侧的每个 API（ConnectivityManager/TrafficStats/私有 DNS 设置）都能在这三段里找到实体。

第二章 网络请求与评分：怎么选默认网

设计洞察 · 学习目标

1. 解释请求侧 NetworkRequest 与供给侧 NetworkFactory 的匹配流程；2. 描述 FullScore 与 NetworkRanker 如何决定默认网；3. 说明 linger 宽限期对应用的意义与默认时长；4. 追踪 setProfileNetworkPreference 的企业分流量场景



图 2 请求-评分时序：NetworkRequest→NetworkFactory→NetworkAgentInfo→FullScore→Ranker→linger 切换

2.1 逐段注释（真实锚点）

java packages/modules/Connectivity/…（骨架+讲解注释）

```
// ConnectivityService.java:314（11435 行！Mainline 后的巨石仍在这里）
public class ConnectivityService extends IConnectivityManager.Stub {
// 讲解注释①：requestNetwork(:1678 createNetworkRequest)：
// App 声明"我要什么"（能力：INTERNET+NOT_METERED+…）；
// 供给侧 NetworkFactory（wifi/cell/eth 各自一个）声明"我有什么"。
// 讲解注释②：NetworkAgentInfo（connectivity/NetworkAgentInfo.java）：
// 一个"活网络"的运行时视图：agent+FullScore+linkProperties+验证状态。
// 每次网络状态变化，Nai 更新并触发 rematch。
// 讲解注释③：NetworkRanker（connectivity/NetworkRanker.java）：
// 对每个请求，在满足其能力的网络里选 FullScore 最高者。
// 讲解注释④：切换的缓冲——LingerMonitor：老网进入 linger 宽限期
```

```
// （默认 30s），期间 App 的 bindSocket 还连着老网；宽限后真断。
// "Wi-Fi 闪断瞬间 App 没断"的秘密。
}
```

先修要求与学习路线

先修：已读第一册（架构总览）。

读完本书你将能够：

- 解释网络栈的核心机制与设计决策
- 追踪关键调用链并标注每次跨进程跳
- 使用 dumpsys/日志定位本领域问题
- 回答书7 对应章节的面试题

> **设计洞察 · 逐段精读：为什么 CS 有一万一千行**
> 它同时是：请求注册中心（本节）、网络评分裁判（Ranker）、Airplane 模式、代理（ProxyTracker）、Keepalive（TcpKeepaliveController）、多网并发（profile preference）……Mainline 化后这团复杂度可独立升级（com.android.tethering APEX），但**没有拆小**——网络状态的全局一致性比代码切分更稀缺。

2.3 多网络并发与 profile（企业/多场景）

13 的 ProfileNetworkPreferenceList（CS/connectivity/ 目录）：按用户 profile 指定"办公流量走企业 APN、其他走默认"——**同一进程内两个网络身份并存**。App 侧 bindSocketToNetwork 显式选网（多开下载器的场景）。读码入口 `ConnectivityService#setProfileNetworkPreference`；与 per-app VPN（第六章）是两种"分流量"手段（网络级 vs 隧道级）。

2.2 观测与排障

`dumpsys connectivity`（NetworkAgentInfo 全表：分数/验证/linger）；`dumpsys connectivity airplane`（飞行模式）；网切日志：logcat ConnectivityService 标签的 rematch 记录。经典题："Wi-Fi 与蜂窝同时开为什么走 Wi-Fi"——TRUSTED+NOT_METERED 加权后 FullScore 稳赢。

第三章 验证：这网能不能信

> **设计洞察 · 学习目标**
> 1. 说出四探针各测什么与失败语义；2. 区分 CAPTIVE_PORTAL 与 PARTIAL_CONNECTIVITY 的判例；3. 解释验证为何异步、结论如何回流评分；4. 在 dumpsys 输出中找到探针结论与重定向 URL



图 3 NetworkMonitor 状态机与四探针（:225 StateMachine、:29-33 PROBE 常量），强制门户与部分连通的下游影响

`NetworkMonitor`（NetworkStack/…/NetworkMonitor.java:225，extends StateMachine——frameworks 的"小 Handler 状态机"库）对每个网络跑四探针：PROBE_DNS/HTTP/HTTPS/PRIVDNS（:29-33）。判例集：302 重定向→CAPTIVE_PORTAL（弹登录页）；HTTPS 不通 HTTP 通→PARTIAL_CONNECTIVITY（"无法访问互联网"提示但 fallback 可用）。验证**异步**：先给网后验网，结论回来重新评分。面试高频："连上 Wi-Fi 提示无法上网但仍能用"= PARTIAL 语义。

第四章 netd 与 eBPF：流量从记账到限速

设计洞察 · 学习目标

1. 说清 netd 在系统中的三大职责；2. 描述 setThreadStatsTag 到 per-uid 计数的全链（含 map 名）；3. 解释 iptables 与 eBPF 的分工边界；4. 用 dumpsys netd 观察规则与计数 map



图 4 eBPF 流量记账链：setThreadStatsTag→tagSocket→cookie map→uid 计数→读取，及 Tethering offload

4.1 数据链逐段

text system/netd/server（结构+讲解注释）

```
main.cpp:111 int main()
// 讲解注释①：netd 承担 CS 的 native 执行：DNS resolver、
// 带宽/防火墙规则（BandwidthController.cpp 同目录）、
// socket tag（BpfNetMaps.java:37 是 Java 门面，native 在 netd）。
// 讲解注释②：eBPF 记账：
// App: TrafficStats.setThreadStatsTag(0xF00D)
// → 新 socket 创建时 tagSocket：内核 eBPF 程序按 socket cookie
// 写 uid_tag_map（uid+tag ↔ cookie）
// → 每包经过：cgroup hook 查 cookie → 累加到 per-uid-tag 计数 map
// → TrafficStats.getUidRxBytes → binder → 读 map 快照
// 讲解注释③：iptables 只剩隔离（UID 禁网），记账全 eBPF：
// map 查 O(1)、可按任意维度聚合——这是 Settings 流量页的后端。
```

设计洞察 · 逐段精读：tag 为什么是 int

TrafficStats 的 tag 是 32 位（应用可自定高 16 位做子类统计，低 16 位是 uid 校验）——同一 App 内区分"图片流量/广告流量"的正统做法。面试追问"防作弊吗"——不防：tag 是 App 自报的，但 uid 部分内核核对（伪造 tag 逃不掉 uid 账）。

4.2 Tethering 的 BPF offload

`IpServer`（Tethering/…/IpServer.java）：每个热点下游接口一个实例（DHCP/NAT/限速）；**BpfCoordinator** 做下行 offload——包直走硬件转发不上 CPU（省电主战场）。观测 `dumpsys connectivity tether`。

第五章 DNS：Private DNS 三态

设计洞察 · 学习目标

1. 说出 Private DNS 三态与各自的失败语义差异；2. 解释 DNS 为何必须经 dnsproxyd 进 netd/resolver；3. 区分系统 DoT 与应用层 DoH；4. 用 dumpsys dnsresolver 观察缓存与服务器状态



图 5 DNS 解析链：bionic→dnsproxyd→netd resolver，DoT 三态与排障三题

三态（DnsManager.java:24-26 常量）：OFF（明文 53）/STRICT（DoT：TLS 853 到指定主机，失败即断不解）/AUTO（先试 TLS 失败回落）。**App 不直接发 DNS**——bionic getaddrinfo 走 `/dev/socket/dnsproxyd` 到 netd：多网络按调用方绑定网选 DNS 服务器，只有系统知道该用哪张网。排障 `dumpsys dnsresolver`；STRICT 配错主机=整体解析失败（不回落）。

第六章 VPN：tun 设备与路由劫持

设计洞察 · 学习目标

1. 描述 establish 从 tun fd 到 NetworkAgent 注册的五步；2. 解释路由劫持如何实现全量代理；3. 说明 protect() 防环原理与缺失后果；4. 区分 always-on 与 lockdown 两种开关语义



图 6 VPN 机制：establish(:1504) 建 tun→路由劫持→protect 防环，平台 VPN 与 always-on/lockdown

text frameworks/.../connectivity/Vpn.java:179/1504（结构+讲解注释）

```
// 讲解注释①：establish(:1504)：TUNSETIFF 建 tun0，返回
// ParcelFileDescriptor ——fd 跨进程（书3 BINDER_TYPE_FDA）
// 讲解注释②：路由劫持：addRoute 的 CIDR 写内核路由（默认 0/0 全量）
// 讲解注释③：protect()：加密通道 socket 标记"绕开 tun"——防递归环
//  （VPN 流量再进 tun=自己套自己）
// 平台 VPN：VpnProfile/IkeSessionParams——密钥协商框架管，App 只拿 fd
// always-on/lockdown：开机自启/断 VPN 即断网（不回落明文）
```

数据面一句话：**VPN App 是用户态路由器**（read tun→加密→protect socket 发物理网→回程 write tun），性能瓶颈在加解密与两次上下文切换。

第七章 设计决策八问

- **为什么网络全 Mainline 化？** 运营商/安全补丁需要急速分发（拖整机 OTA 等不起）；接口（NetworkAgent AIDL）稳定是前提。
- **为什么验证异步不阻塞？** 可用性优先：部分连通先顶上，结论回来再调整——比"等验完才给网"体验好。
- **为什么 DNS 走 socket 不走 binder？** bionic 是 C 库（无 binder C API）；且 fork 出的进程禁 binder（书2 §六 呼应）。
- **为什么记账用 eBPF？** O(1) map 查询+任意维度聚合；iptables 计数 O(rules) 且写规则慢。
- **为什么 STRICT 失败不回落？** 用户明示"只要加密"——回落即违背意愿（安全语义 > 可用性）。
- **为什么 VPN protect 必须显式？** 隐式判断"哪些 socket 是隧道"不可靠（框架无法识别加密流量的目的）；责任交给最知情的 VPN App。

- **为什么 linger 不立即断老网？** 切换抖动（Wi-Fi 闪断）频繁重连代价高；宽限期吸收毛刺。
- **为什么 NetworkMonitor 独立成 NetworkStack 模块？** 验证逻辑与 captive portal 协议高频演进；且被多模块复用（Wi-Fi/CS/Tethering 都要问"这网通吗"）。

附录 A 观测实操

目标	命令	看点	
网络全表	dumpsys connectivity	NetworkAgentInfo：分数/验证/linger	
验证历史	dumpsys connectivity |	grep -A5 Validated	探针结论
DNS	dumpsys dnsresolver	缓存命中/服务器/Private DNS 模式	
eBPF/流量	dumpsys netd	tag map/规则	
热点	dumpsys connectivity tether	下游客户端/offload	
VPN	dumpsys connectivity |	grep -A5 VPN	tun/uid 范围/always-on
路由	ip route show table all	tun/多网路由表	

附录 B 快问快答（16 条）

- 本树 CS 在哪？——packages/modules/Connectivity/（Mainline）。
- NetworkMonitor 在哪？——packages/modules/NetworkStack/（独立模块）。
- Vpn.java 为何没走？——frameworks/base/services/core/.../connectivity/Vpn.java:179（活化石）。
- 四探针是哪四个？——DNS/HTTP/HTTPS/PRIVDNS（:29-33）。
- PARTIAL 什么语义？——HTTPS 不通但 HTTP 通：fallback 可用。
- 默认网怎么选？——Ranker 在满足能力的网里挑 FullScore 最高。
- linger 多久？——默认 30s（吸收切换毛刺）。
- tagSocket 写到哪个 map？——uid_tag_map（cookie→uid+tag）。
- TrafficStats tag 谁核对？——uid 部分内核核对，tag 高 16 位 App 自由。
- iptables 现在管什么？——UID 隔离/限速；记账已 eBPF。
- Private DNS STRICT 失败？——整体解析失败（不回落明文）。
- DoH 系统支持吗？——默认 DoT（853）；DoH 是 App 库层行为。
- VPN 防环靠什么？——protect() 标记隧道 socket 绕 tun。

- always-on 与 lockdown 区别？——自启重连 vs 无 VPN 即断网。
- Tethering offload 价值？——下行硬件转发不上 CPU（省电）。
- 学完往哪走？——书11 存储加密（data 完整性的另一半）。

附录 C 自检题

- 默画"请求→评分→默认网→linger"时序；Wi-Fi/蜂窝分差翻转时 App 感知什么。
- 四探针各测什么？302 与 HTTPS 超时分别判什么？
- 一个 socket 的流量从 tag 到被 Settings 统计的全链（含 map 名）。
- Private DNS 三态与失败语义；为什么 DNS 必须过 netd。
- VPN 数据面一个包的双程；protect 缺失会发生什么。

第八章 ConnectivityService 决策内核：注册与切网全链

设计洞察 · 学习目标

1. 复述 registerNetworkAgent 的两阶段注册与 NAI 真正入表的时机；2. 说出 rematch→processDefaultNetworkChanges→makeDefault 的完整调用链；3. 解释 makeDefault 对代理/TCP 缓冲/记账/linger 的联动面；4. 追踪 mNetd.networkSetDefault 落到内核路由的最后一跳



图 7 请求-评分时序：NetworkRequest→NetworkFactory→NetworkAgentInfo→FullScore→Ranker→linger 切换

第二章给了"谁当默认网"的鸟瞰：请求侧（NetworkRequest）声明要什么、供给侧（NetworkFactory）声明有什么、Ranker 择优。本章走进决策内核的两条关键函数链：**网络如何出生**（registerNetworkAgent，CS.java:7283）、**默认网如何易主**（rematch→makeDefault，CS.java:8539-8612）。所有锚点位于 packages/modules/Connectivity/service/src/com/android/server/ConnectivityService.java（下称 CS，Mainline 模块，11435 行）。

8.1 网络的出生证：registerNetworkAgent

java packages/modules/Connectivity/service/src/com/android/server/ConnectivityService.java:7283-7341（骨架+讲解注释）

```
// 讲解注释①：这是 AIDL 服务端方法——Wi-Fi/Telephony/Ethernet 的 NetworkAgent 所在进程
// 经 binder 调入（NetworkAgent.register() → ConnectivityManager.registerNetworkAgent，
// framework/NetworkAgent.java:722）。INetworkAgent na 是对端进程状态机的代理对象。
public Network registerNetworkAgent(INetworkAgent na, NetworkInfo networkInfo,
LinkProperties linkProperties, NetworkCapabilities networkCapabilities,
@NonNull NetworkScore initialScore, NetworkAgentConfig networkAgentConfig,
int providerId) {
Objects.requireNonNull(networkInfo, "networkInfo must not be null"); // 五要素一个不能为空
...
if (networkCapabilities.hasTransport(TRANSPORT_TEST)) {
// 讲解注释②：权限关口分岔：TEST 网络走 MANAGE_TEST_NETWORKS；
// 其余必须持 NETWORK_FACTORY——普通 App 冒充不了网络供给方。
enforceAnyPermissionOf(Manifest.permission.MANAGE_TEST_NETWORKS);
} else {
enforceNetworkFactoryPermission();
}
final int uid = mDeps.getCallingUid();
final long token = Binder.clearCallingIdentity(); // 讲解注释③：清 binder 身份——
try { // 后续注册动作以系统进程身份执行，
return registerNetworkAgentInternal(na, networkInfo, linkProperties,
networkCapabilities, initialScore, networkAgentConfig, providerId, uid);
} finally {
Binder.restoreCallingIdentity(token); // finally 里恢复，异常也不泄漏身份
}
}
private Network registerNetworkAgentInternal(INetworkAgent na, ...) {
// 讲解注释④：构造 NAI——reserveNetId() 分配 netId（内核 per-network 路由表编号，
// 对应 ip rule 里的表号）；NAI 同时持有 mNetd/mDnsResolver：一个"活网络"的
// 运行时视图就此就位（agent+分数+linkProperties+验证状态）。
final NetworkAgentInfo nai = new NetworkAgentInfo(na,
new Network(mNetIdManager.reserveNetId()), new NetworkInfo(networkInfo),
linkProperties, networkCapabilities,
currentScore, mContext, mTrackerHandler, new NetworkAgentConfig(networkAgentConfig),
this, mNetd, mDnsResolver, providerId, uid, mLingerDelayMs,
mQosCallbackTracker, mDeps);
...
if (DBG) log("registerNetworkAgent " + nai);
// 讲解注释⑤：关键解耦——NetworkMonitor 不在本进程 new，而是经 NetworkStack
// 客户端到独立模块进程创建（第三章的四探针实体），回调经
// NetworkMonitorCallbacks 走 binder 返回 CS。模块可独立升级。
mDeps.getNetworkStack().makeNetworkMonitor(
nai.network, name, new NetworkMonitorCallbacks(nai));
```

```
// 讲解注释⑥：此时注册尚未完成！真正的收尾在 handleRegisterNetworkAgent(:7345)：
// 若网络在此前断连/发事件，消息被 NetworkAgent 暂存，等 nai.connect() 后重放。
return nai.network; // "准生证"：netId 有了，但网络还没进入 rematch 池
}
```

设计洞察 · 逐段精读：两阶段注册

registerNetworkAgent 返回时 NAI 还没入表：handleRegisterNetworkAgent(:7345) 才做 mixInCapabilities（把 agent 自报的能力做安全净化，如 ACCESS_UID 检查）、put 进 mNetworkAgentInfos 与 mNetworkForNetId、启动 NetworkMonitor。这解释了 dumpsys connectivity 里"网络已出现但验证状态为空"的窗口期：agent 已连接、监视器未就位，事件不会丢只会迟。

8.2 默认网易主：rematch 与 makeDefault

rematch 的产出是一个 NetworkReassignment 事件集（computeNetworkReassignment，CS.java:8802：对每个请求算出 bestNetwork，与旧满足者比较），applyNetworkReassignment（:8874）再按序落地——**先更新各网满足的请求清单，再处理默认网变化，再发回调，最后处理 linger 与拆除**。默认网变化这条支线如下。

java packages/modules/Connectivity/service/src/com/android/server/ConnectivityService.java:8519-8612（骨架+讲解注释）

```
private void processDefaultNetworkChanges(@NonNull final NetworkReassignment changes) {
boolean isDefaultChanged = false;
for (final NetworkRequestInfo defaultRequestInfo : mDefaultNetworkRequests) {
final NetworkReassignment.RequestReassignment reassignment =
changes.getReassignment(defaultRequestInfo);
if (null == reassignment) continue; // 该请求的满足者没变就跳过
isDefaultChanged = true;
// 讲解注释①：reassignment 只含"满足者变了"的请求；系统默认请求与
// per-app 默认请求（多网络并发，见 2.3）都从这里分发。
makeDefault(defaultRequestInfo, reassignment.mOldNetwork, reassignment.mNewNetwork);
}
if (isDefaultChanged) {
// 讲解注释②：切换瞬间申请一台短时 wakelock——给 App 一小段有电时间迁移连接，
// 避免休眠中的设备切到一半断流。
scheduleReleaseNetworkTransitionWakelock();
}
}
private void makeDefault(@NonNull final NetworkRequestInfo nri, ...) {
if (newDefaultNetwork != null) {
// 讲解注释③：VPN 是"网络之上的网络"（第六章）：默认网换了，VPN 的
// underlyingNetworks 能力要跟着重算，否则上层看到的能力是旧的。
propagateUnderlyingNetworkCapabilities(newDefaultNetwork.network);
}
if (mDefaultRequest != nri) { // per-app 默认网：不走内核默认路由，
makeDefaultForApps(nri, oldDefaultNetwork, newDefaultNetwork); // 走 netd 的 UID 规则
return; //  （networkAddUidRangesParcel 把 UID 段绑到新网）
}
makeDefaultNetwork(newDefaultNetwork); // 讲解注释④：系统默认网→netd.networkSetDefault
if (oldDefaultNetwork != null) {
// 讲解注释⑤：老默认网进入 linger 宽限（默认 30s，2.1 的 LingerMonitor）——
// "Wi-Fi 闪断 App 不断"的机制落点就在这一行。
```

```
mLingerMonitor.noteLingerDefaultNetwork(oldDefaultNetwork, newDefaultNetwork);
}
mNetworkActivityTracker.updateDataActivityTracking(newDefaultNetwork, oldDefaultNetwork);
handleApplyDefaultProxy(null != newDefaultNetwork
? newDefaultNetwork.linkProperties.getHttpProxy() : null); // 代理随网切换
updateTcpBufferSizes(null != newDefaultNetwork
? newDefaultNetwork.linkProperties.getTcpBufferSizes() : null); // TCP 缓冲按网调优
notifyIfacesChangedForNetworkStats(); // 讲解注释⑥：通知记账模块重读 iface（第四章）
}
private void makeDefaultNetwork(@Nullable final NetworkAgentInfo newDefaultNetwork) {
try {
if (null != newDefaultNetwork) {
// 讲解注释⑦：一行就是"内核里的默认网"：netd 收到后改写默认路由与
// 路由规则（per-netId 表+主表选择器）——netd 对接详见第四章。
mNetd.networkSetDefault(newDefaultNetwork.network.getNetId());
} else {
mNetd.networkClearDefault(); // 没有可用网络时清空默认路由
}
} catch (RemoteException | ServiceSpecificException e) {
loge("Exception setting default network :" + e);
}
}
```

设计洞察 · 设计洞察：makeDefault 是一套联动开关

一次默认网切换联动了六件事：内核路由（networkSetDefault）、VPN 底层能力传播、per-app UID 规则、代理（ProxyTracker）、TCP 缓冲（蜂窝小缓冲/Wi-Fi 大缓冲）、记账 iface 通知。漏掉任何一环就是"切网后代理失效/流量算错账/TCP 吞吐异常"这类经典 bug——读码时把 makeDefault 当 checklist，改它时当回归清单。

8.3 尾声：linger 超时与网络拆除

切换不是终点：老网 linger 到期后 handleLingerComplete（CS.java:8499）调用 unneeded（:4601，按"是否还满足任何前台请求"判定），为真则 teardownUnneededNetwork（:8486）→nai.disconnect()，agent 收到断开命令走 netd 清路由；为假（网络被别的请求重新需要，如"仅 Wi-Fi 下载"恢复）则 updateCapabilitiesForNetwork 把它降为后台网。**一网一生**：出生（8.1）→服役（rematch 供请求）→退役（linger→teardown），三段全在这 11435 行里闭环。

与第三章交叉：makeNetworkMonitor 创建的验证结论经 NetworkMonitorCallbacks 回流，改变 NAI 的 validity 与 FullScore，正是 rematch 的输入之一；与第六章交叉：VPN 的 establish 在拿到 tun fd 后同样走 registerNetworkAgent 注册成网——决策内核对"真网络"与"虚拟网络"一视同仁。

第九章 NetworkMonitor 评估状态机全解

设计洞察 · 学习目标

1. 画出 Validated/Evaluating/Probing/CaptivePortal/EvaluatingPrivateDns/WaitingForNextProbe 六态的转移图；2. 解释 token 机制如何隔离新旧探针线程的结果；3. 说出 PARTIAL_CONNECTIVITY 的判定与用户接受后的状态变化；4. 描述 EvaluationState 位掩码如何向 CS 汇报探针进度



图 8 NetworkMonitor 状态机与四探针（:225 StateMachine、:29-33 PROBE 常量），强制门户与部分连通的下游影响

第三章给出了 NetworkMonitor（packages/modules/NetworkStack/src/com/android/server/connectivity/NetworkMonitor.java，3749 行，extends StateMachine——frameworks 的单线程 Handler 状态机库）的结论集；本章逐段走读状态机本体。每个网络一份 NetworkMonitor 实例，由 CS 的 makeNetworkMonitor（第八章 8.1）在 NetworkStack 进程创建。

9.1 六态地图

状态	锚点	职责	去向
ValidatedState	NM.java:1115	验证通过的常驻态：周期 TCP 轮询、DNS 停滞检测	可疑→EvaluatingState 重评估
EvaluatingState	NM.java:1335	评估入口：发 CMD_REEVALUATE、清探针结果	进 Probing 或跳过验证
ProbingState	NM.java:1665	子线程并行执行四探针，等待汇总	成功/门户/部分/失败四岔
CaptivePortalState	NM.java:1485	门户登录态：通知拉起 portal App、定时 RECHECK	登录成功→重评估

状态	锚点	职责	去向
EvaluatingPrivateDnsState	NM.java:1520	Private DNS 服务器解析与验证（第五章）	完成后→ValidatedState
WaitingForNextProbeState	NM.java:1771	失败退避：指数退避等下一轮（初始延迟逐轮加码）	延迟到→回 EvaluatingState

9.2 评估入口：EvaluatingState

java packages/modules/NetworkStack/src/com/android/server/connectivity/NetworkMonitor.java:1335-1462（骨架+讲解注释）

```
private class EvaluatingState extends State {
private Uri mEvaluatingCapportUrl;
@Override
public void enter() {
// 讲解注释①：计时器只在首轮启动——反复进出本态（重评估）不重置，评估总时长才真实。
if (!mEvaluationTimer.isStarted()) mEvaluationTimer.start();
// 讲解注释②：Terms&Conditions 型门户（captivePortalData 携带运营商配置的门户
URL）首次评估直接判门户，不走 HTTP 探针。
if (isTermsAndConditionsCaptive(mInfoShim.getCaptivePortalData(mLinkProperties))
&& mValidations == 0) {
mLastPortalProbeResult = new CaptivePortalProbeResult(PORTAL_CODE, ...);
mEvaluationState.reportEvaluationResult(NETWORK_VALIDATION_RESULT_INVALID, ...);
transitionTo(mCaptivePortalState);
return;
}
sendMessage(CMD_REEVALUATE, ++mReevaluateToken, 0); // 讲解注释③：token 化的评估令牌——
if (mUidResponsibleForReeval != INVALID_UID) { // 旧令牌的消息一律作废
TrafficStats.setThreadStatsUid(mUidResponsibleForReeval); // 探针流量记到"肇事者"头上
mUidResponsibleForReeval = INVALID_UID;
}
mReevaluateDelayMs = INITIAL_REEVALUATE_DELAY_MS; mEvaluateAttempts = 0; // 退避起点
mEvaluatingCapportUrl = getCaptivePortalApiUrl(mLinkProperties); // 记住 capport API URL，
// 讲解注释④：清探针位掩码但保留验证结论——用于对比"这轮比上轮好了多少"。
mEvaluationState.clearProbeResults();
}
@Override
public boolean processMessage(Message message) {
switch (message.what) {
case CMD_REEVALUATE:
if (message.arg1 != mReevaluateToken || mUserDoesNotWant) {
return HANDLED; // 讲解注释③的兑现：过期 token 直接吞掉
}
// 讲解注释⑤：不值得验证的网络直接跳过：VPN（用户显式选择，意愿高于探针）、
// 不提供 INTERNET 的网、不可信网；但 PrivDNS 校验需求独立存在（STRICT 仍要解析）。
if (mEvaluationState.isCaptivePortalWantedAsIs()
&& isPrivateDnsValidationRequired()) {
transitionTo(mEvaluatingPrivateDnsState);
return HANDLED;
} else if (!isValidationRequired()) {
transitionTo(isPrivateDnsValidationRequired() ? mEvaluatingPrivateDnsState : mVa
lidatedState); // 跳过探针
return HANDLED;
```

```
}
mEvaluateAttempts++; transitionTo(mProbingState); // 正式进入探针态
return HANDLED;
case EVENT_ACCEPT_PARTIAL_CONNECTIVITY:
// 讲解注释⑥：用户点了"仍使用此网络"：关掉 HTTPS 探针（再测只会反复报 PARTIAL），转 PrivDNS
校验收尾。
maybeDisableHttpsProbing(true /* acceptPartial */);
transitionTo(mEvaluatingPrivateDnsState);
return HANDLED;
default:
return NOT_HANDLED;
}
}
@Override
public void exit() {
TrafficStats.clearThreadStatsUid(); // 讲解注释⑦：探针 UID 归属用完即清
}
}
```

设计洞察 · 逐段精读：谁在替用户付探针流量

门户口网络里用户点过"登录此网"后（CMD_FORCE_REEVALUATE 携带 UID），后续重评估的探针流量经 setThreadStatsUid 记到该 UID——流量会计上"验证是我发起的"就记我的账（TrafficStats 机制见第四章）。评估退出时 clearThreadStatsUid，避免把下一轮普通探针也算到无辜者头上。

9.3 探针执行与判定：ProbingState 与 isCaptivePortal

java packages/modules/NetworkStack/src/com/android/server/connectivity/NetworkMonitor.java:1665-1760 与 2445-2513（骨架+讲解注释）

```
private class ProbingState extends State {
private Thread mThread;
@Override
public void enter() {
...
final int token = ++mProbeToken; // 讲解注释①：新探针周期领新 token
final URL fallbackUrl = nextFallbackUrl(); // 备用探针 URL 轮转（多个 fallback 轮流用）
final URL[] httpsUrls = Arrays.copyOf(mCaptivePortalHttpsUrls, ...), httpUrls = Arrays.copyO
f(mCaptivePortalHttpUrls, ...);
// 讲解注释②：状态机线程不做网络 IO——探针在子线程跑完，结果打包成 CMD_PROBE_COMPLETE
消息回灌状态机（超时 3s，PROBE_TIMEOUT_MS:242）
。
mThread = new Thread(() -> sendMessage(obtainMessage(CMD_PROBE_COMPLETE, token, 0,
isCaptivePortal(deps, httpsUrls, httpUrls, fallbackUrl))));
mThread.start();
}
@Override
public boolean processMessage(Message message) {
switch (message.what) {
case CMD_PROBE_COMPLETE:
if (message.arg1 != mProbeToken) return HANDLED; // 讲解注释③：过期线程的结果丢弃
final CaptivePortalProbeResult probeResult = (CaptivePortalProbeResult) message.obj;
if (probeResult.isSuccessful()) {
// 讲解注释④：204 到手：转 PrivDNS 校验态收尾（无 PrivDNS 需求时直通 Validated）
```

```
transitionTo(mEvaluatingPrivateDnsState);
} else if (probeResult.isPortal()) {
mLastPortalProbeResult = probeResult; // 记下重定向 URL 给 portal App
transitionTo(mCaptivePortalState);
} else if (probeResult.isPartialConnectivity()) {
// 讲解注释⑤：HTTPS 不通 HTTP 通=部分连通；用户未接受则进退避态等下一轮
mEvaluationState.reportEvaluationResult(NETWORK_VALIDATION_RESULT_PARTIAL, null)
;
if (mAcceptPartialConnectivity) transitionTo(mEvaluatingPrivateDnsState);
else transitionTo(mWaitingForNextProbeState);
} else {
mEvaluationState.reportEvaluationResult(NETWORK_VALIDATION_RESULT_INVALID, null)
;
transitionTo(mWaitingForNextProbeState); // 讲解注释⑥：全失败→指数退避
}
return HANDLED;
default:
deferMessage(message); // 讲解注释⑦：探针期间来的消息延后到下一态再处理
return HANDLED;
}
}
}
private CaptivePortalProbeResult isCaptivePortal(ValidationProperties properties,
URL[] httpsUrls, URL[] httpUrls, URL fallbackUrl) {
if (!mIsCaptivePortalCheckEnabled) return CaptivePortalProbeResult.success(...);
final ProxyInfo proxyInfo = mLinkProperties.getHttpProxy();
if (proxyInfo != null && !Uri.EMPTY.equals(proxyInfo.getPacFileUrl())) {
// 讲解注释⑧：PAC 代理网不 fetch generate_204 而是拉 PAC 脚本本身——204 探针可能被 PAC
拦截导致假阴性（源码注释里三个理由，值得精读）。
pacUrl = makeURL(proxyInfo.getPacFileUrl().toString());
}
if (pacUrl != null) { // PAC 路径
result = sendDnsAndHttpProbes(null, pacUrl, ValidationProbeEvent.PROBE_PAC);
} else if (mUseHttps && ...) { // 讲解注释⑨：主路径：HTTP/HTTPS 并行+fallback
result = sendHttpAndHttpsParallelWithFallbackProbes(properties, proxyInfo,
httpsUrls[0], httpUrls[0], fallbackUrl);
// 判例：HTTPS 收 3xx/内容不符→portal；HTTPS 超时而 HTTP 204→PARTIAL；HTTP 收
302→portal（redirectUrl 即登录页）
。
} else { // 降级路径：只测 HTTP
result = sendDnsAndHttpProbes(proxyInfo, httpUrls[0], ValidationProbeEvent.PROBE_HTTP);
}
return result;
}
```

9.4 结果的记账本：EvaluationState

探针结果不散落在成员变量里，而是收敛到 EvaluationState（NM.java:3530）三个位掩码：mProbeCompleted（哪些探针跑完了）、mProbeResults（哪些成功）、mEvaluationResult（VALID/INVALID/PARTIAL 的最终结论）。每次 noteProbeResult 变化即 notifyProbeStatusChanged 上报 CS——**Settings 里的"已连接/可用性受限"实时刷新**靠的就是这条增量上报；reportEvaluationResult 再打一个 NetworkTestResultParcelable（含 redirectUrl、时间

戳）给 CS，CS 把 VALID 位写进 FullScore（第二章）触发 rematch。

设计洞察 · 设计洞察：PARTIAL 为什么值得单独一态

HTTPS 不通 HTTP 通，严格说"网是坏的"（加密流量不可用），但完全可用作 fallback。平台选择三步走：先如实上报 PARTIAL（通知栏"无法访问互联网但可连"）；等用户表态（接受→置 VALID+PARTIAL 并关 HTTPS 探针，避免无限循环重测）；若用户不理，WaitingForNextProbeState 指数退避继续测。安全语义（HTTPS 可用性）与可用性（总比断网强）在此显式权衡——这是状态机里最值得面试展开的一段。

第十章 eBPF 数据面：cgroup 计账程序与热点卸载

设计洞察 · 学习目标

1. 说出 bpf_traffic_account 从 skb 到 per-uid 计数的五步；2. 背出计账核心 map（cookie_tag_map/stats_map_A/app_uid_stats_map）各自的角色；3. 解释 stats_map_A/B 双 map 轮换要解决的读写竞争；4. 描述 BpfCoordinator 每 5 秒从 TetherStatsValue 汇总热点流量的链路



图 9 eBPF 流量记账链：setThreadStatsTag→tagSocket→cookie map→uid 计数→读取，及 Tethering offload

第四章给了记账链的鸟瞰：App 打 tag→tagSocket 写 cookie map→每包 cgroup hook 计数→TrafficStats 读数。本章下沉到**内核侧程序本体**与**热点侧卸载统计**：前者在 packages/modules/Connectivity/bpf_progs/netd.c（450 行，T 上 Connectivity APEX 自带 bpf 程序与 bpfloader 配置），后者在 Tethering 的 BpfCoordinator。

10.1 每包五步：bpf_traffic_account

c packages/modules/Connectivity/bpf_progs/netd.c:282-356（骨架+讲解注释）

```
// 讲解注释①：挂载点 cgroupskb/ingress 与 egress（:347/:352）—— attaching 到
// root cgroup 后，设备上每个 socket 收发的每个包都会经过这里一次。
// skb（socket buffer 元数据）里拿不到应用身份，身份要从三层来源拼。
static __always_inline inline int bpf_traffic_account(struct __sk_buff* skb, int direction) {
uint32_t sock_uid = bpf_get_socket_uid(skb); // 步骤1：内核 socket 属主 uid
uint64_t cookie = bpf_get_socket_cookie(skb); // 步骤2：socket 生命周期唯一 cookie
UidTagValue* utag = bpf_cookie_tag_map_lookup_elem(&cookie);
uint32_t uid, tag;
if (utag) {
uid = utag->uid; tag = utag->tag; // 步骤3：有 tag 记录→用 App 自报身份
} else {
uid = sock_uid; tag = 0; // 没 tag→回落内核 uid、tag=0
}
// 讲解注释②：clat 特判：464xlat 的原始 IPv6 流量不计账——同一份数据在
// 虚拟 v4 接口上还会再计一次，这里放行是为了防"双计"（第三章 Nat464Xlat）。
if (sock_uid == AID_CLAT || uid == AID_CLAT) return BPF_PASS;
int match = bpf_owner_match(skb, sock_uid, direction); // 步骤4：防火墙判决（uid_owner_map
if ((direction == BPF_EGRESS) && (match == BPF_DROP)) { // ：后台禁网/省流模式等规则，
return match; // 讲解注释③：出向要丢的包不再计数——"没发出去不算流量"
}
// 讲解注释④：DNS 特判：系统 DNS（uid=AID_DNS, tag=TAG_SYSTEM_DNS）的流量
// 记回归属 App 而非记到 netd 头上——用户查"谁在偷跑 DNS"时账能对上。
if (tag == TAG_SYSTEM_DNS && uid == AID_DNS) { uid = sock_uid; ... }
StatsKey key = {.uid = uid, .tag = tag, .counterSet = 0, .ifaceIndex = skb->ifindex};
// 步骤5：拼计数键——uid+tag+counterSet（前后台，uid_counterset_map:76 查得）+
// ifaceIndex（按网口分开），然后按 configuration_map 选中的活 map 累加。
uint8_t* counterSet = bpf_uid_counterset_map_lookup_elem(&uid);
if (counterSet) key.counterSet = (uint32_t)*counterSet;
uint32_t mapSettingKey = CURRENT_STATS_MAP_CONFIGURATION_KEY;
uint32_t* selectedMap = bpf_configuration_map_lookup_elem(&mapSettingKey);
if (!selectedMap) { asm("%0 &= 1" : "+r"(match)); return match; } // verifier 辅助
if (key.tag) { update_stats_with_config(skb, direction, &key, *selectedMap); key.tag = 0; }
update_stats_with_config(skb, direction, &key, *selectedMap); // tag 账+无 tag 账各记一遍
update_app_uid_stats_map(skb, direction, &uid); // 步骤5b：per-uid 总账另记一份
asm("%0 &= 1" : "+r"(match));
return match;
}
DEFINE_NETD_BPF_PROG("cgroupskb/ingress/stats", AID_ROOT, AID_SYSTEM, bpf_cgroup_ingress)
(struct __sk_buff* skb) { return bpf_traffic_account(skb, BPF_INGRESS); }
DEFINE_NETD_BPF_PROG("cgroupskb/egress/stats", AID_ROOT, AID_SYSTEM, bpf_cgroup_egress)
(struct __sk_buff* skb) { return bpf_traffic_account(skb, BPF_EGRESS); }
```

10.2 计账 map 一览

map	锚点	键	值	谁读
cookie_tag_map	netd.c:75	socket cookie	UidTagValue（uid+tag）	eBPF 程序

map	锚点	键	值	谁读
stats_map_A/B	netd.c:78-79	StatsKey（uid+tag+counterSet+iface）	StatsValue（rx/tx 字节包数）	netd 导出
app_uid_stats_map	netd.c:77	uid	StatsValue	netd 导出
uid_counterset_map	netd.c:76	uid	前后台标记	eBPF 程序
uid_owner_map	netd.c:81	uid	放行/禁止规则	eBPF 程序
uid_permission_map	netd.c:82	uid	联网权限位	eBPF 程序

设计洞察 · 逐段精读：stats_map_A/B 双 map 轮换

计数只有写侧（eBPF 每包累加）与读侧（netd 周期导出到 NetworkStatsService）。若只有一张 map，读侧遍历期间写侧还在加，导出的快照永远"正在变形"。平台解法：写侧按 configuration_map 指向写 A 或 B；netd 切换指针后，旧 map 冻结为只读快照供从容导出，读完清零备下次轮换——无锁，代价仅一次 map 指针翻转。这是 eBPF 计账"任意维度聚合 O(1)"之外的第二个设计红利。

10.3 热点卸载侧：BpfCoordinator 的统计轮询

热点（tethering）下行的转发流量不走 per-socket 计账——转发包没有本机 socket。它的账记在 offload.c 的 tc 程序（schedcls/tether_downstream4_ether 等，offload.c:660-667，T 上 v4 转发需内核 5.8+）写入的 per-iface TetherStatsValue 里，由 BpfCoordinator（Tethering/src/com/android/networkstack/tethering/BpfCoordinator.java:105）周期汇总。

java packages/modules/Connectivity/Tethering/src/com/android/networkstack/tethering/BpfCoordinator.java:1869-1926（骨架+讲解注释）

```
private void updateQuotaAndStatsFromSnapshot(
@NonNull final SparseArray<TetherStatsValue> tetherStatsList) {
long usedAlertQuota = 0;
for (int i = 0; i < tetherStatsList.size(); i++) {
final Integer ifIndex = tetherStatsList.keyAt(i); // 每 upstream 口一份统计
final TetherStatsValue tetherStats = tetherStatsList.valueAt(i);
// 讲解注释①：map 里是开机以来的累计值，不是增量——差分才是本期流量
final ForwardedStats curr = new ForwardedStats(tetherStats);
final ForwardedStats base = mStats.get(ifIndex); // 读侧缓存的上一轮基线
final ForwardedStats diff = (base != null) ? curr.subtract(base) : curr;
usedAlertQuota += diff.rxBytes + diff.txBytes;
mStats.put(ifIndex, curr); // 讲解注释②：缓存新基线供下轮差分
// 讲解注释③：差分喂给 NetworkStatsService 的 statsProvider——Settings 数据使用页
// 里"热点"那一栏的数据源（STATS_PER_IFACE 与 STATS_PER_UID 两套口径各一份）。
if (mStatsProvider != null) {
mStatsProvider.accumulateDiff(
buildNetworkStats(StatsType.STATS_PER_IFACE, ifIndex, diff),
buildNetworkStats(StatsType.STATS_PER_UID, ifIndex, diff));
}
}
if (mRemainingAlertQuota > 0 && usedAlertQuota > 0) {
```

```
// 讲解注释④：配额告警：运营商/管理员设定的热点额度被差分消耗，
// 耗尽时 notifyAlertReached() 通知上层（停热点或提醒用户）。
final long newQuota = Math.max(mRemainingAlertQuota - usedAlertQuota, 0);
updateAlertQuota(newQuota);
}
}
private void updateForwardedStats() {
final SparseArray<TetherStatsValue> tetherStatsList =
mBpfCoordinatorShim.tetherOffloadGetStats(); // 讲解注释⑤：一次性读回所有
if (tetherStatsList == null) { mLog.e("Problem fetching tethering stats"); return; }
updateQuotaAndStatsFromSnapshot(tetherStatsList); // upstream 的累计计数
}
// 轮询节奏：getPollingInterval(:1917) 取配置值与
// DEFAULT_TETHER_OFFLOAD_POLL_INTERVAL_MS（TetheringConfiguration.java:129=5000ms）的较大者。
// 清理路径：maybeClearLimit(:1755) 在规则清空后 tetherOffloadGetAndClearStats(:1762) 读走尾差。
```

设计洞察 · 设计洞察：offload 与记账是同一枚硬币的两面

offload 的价值是"包不上 CPU"（硬件直转发省电），代价是内核 socket 层看不见这些包——所以必须另起一套 per-iface 计数（TetherStatsValue）并把差分并回 NetworkStatsService。理解了这一点就明白：热点流量统计"迟 5 秒"不是 bug，是轮询周期的物理下限；而 v4 offload 要求内核 5.8+（offload.c:648-667 的 KVER 门控）则是老设备热点统计路径回退到 CPU 转发的原因。

第十一章 DNS 解析内核：dnsproxyd 协议与 DoT 校验

设计洞察 · 学习目标

1. 复述 bionic 到 resolv 的完整查询路径（含 socket 与协议格式）；2. 解释 GetAddrInfoHandler 为什么 spawn 子线程处理；3. 说出 res_cache 的命中/失败缓存语义；4. 描述 PrivateDnsConfiguration 校验线程与 DnsTlsTransport.validate 的关系



图 10 DNS 解析链：bionic→dnsproxyd→netd resolver，DoT 三态与排障三题

第五章给出了 Private DNS 三态的结论集；本章走进解析内核：bionic 怎么把 getaddrinfo 递进来（代理协议）、DnsResolver 模块怎么处理与缓存（DnsProxyListener.cpp/res_cache.cpp）、DoT 服务器怎么被验证（PrivateDnsConfiguration.cpp/DnsTlsTransport.cpp）。这些 C/C++ 实体位于 packages/modules/DnsResolver/（独立 APEX，与 Connectivity 解耦升级）。

11.1 递进路径：bionic 到 dnsproxyd

text bionic/libc/dns/net/getaddrinfo.c:406-432 与 packages/modules/DnsResolver（跨进程协议+讲解注释）

```
App 进程（任意 C/C++ 程序）：
getaddrinfo("example.com", ...)
→ android_getaddrinfo_proxy（getaddrinfo.c:406）
// 讲解注释①：不是所有查询都进代理：本地可判的（数字地址/localhost 等）
// 直接本地处理；这里只拦截真正要发网络查询的。
FILE* proxy = fdopen(__netdClientDispatch.dnsOpenProxy(), "r+"); // :425
// 讲解注释②：dnsOpenProxy 的实现在 system/netd/client/NetdClient.cpp:311——
// connect 到 /dev/socket/dnsproxyd（init 由 netd.rc 的 socket 语句创建，
// socket 名常量见 DnsProxyListener.h:38 SOCKET_NAME="dnsproxyd"）。
// 用 unix socket 而非 binder：bionic 是纯 C 库，没有 binder 通道，
// 且 fork 出的子进程禁 binder（书2 §六）。
fprintf(proxy, "getaddrinfo %s %s %d %d %d %d %u", // :432——文本行协议：
name, service, flags, family, socktype, protocol, netid) // 一行参数+二进制应答
→ DnsResolver 进程（独立 APEX）：
DnsProxyListener::startListener（DnsProxyListener.cpp:573）
registerCmd(new GetAddrInfoCmd()) // "getaddrinfo" 命令注册
GetAddrInfoCmd::runCommand（:813） // 解析 8 个参数：name/service/四 flags/netId
const uid_t uid = cli->getUid(); // 讲解注释③：socket 凭据读出调用方 uid——
gResNetdCallbacks.get_network_context(netId, uid, &netcontext); // 后面按它选网/限流
(new GetAddrInfoHandler(cli, name, service, move(hints), netcontext))->spawn();
// 讲解注释④：spawn=另起工作线程处理——监听线程立刻回去收下一条查询；
// DNS 解析可能耗时数百毫秒（DoT 握手/超时重试），绝不能堵住 accept 循环。
```

11.2 处理与缓存：GetAddrInfoHandler 与 res_cache

cpp packages/modules/DnsResolver/DnsProxyListener.cpp:729-770（骨架+讲解注释）

```
void DnsProxyListener::GetAddrInfoHandler::run() {
Stopwatch s;
// 讲解注释⓪：Handler 持有 client/name/hints/netcontext——线程里不再碰监听循环的共享状态。
maybeFixupNetContext(&mNetContext, mClient->getPid()); // 把 netId/mark 修正为 DNS 专用视角
const uid_t uid = mClient->getUid();
int32_t rv = 0;
NetworkDnsEventReported event;
initDnsEvent(&event, mNetContext);
if (queryLimiter.start(uid)) {
// 讲解注释①：queryLimiter：每 UID 并发配额（OperationLimiter.h）——应用打爆
// resolver 的防线，超限返回 EAI_MEMORY，客户端表现为本轮解析失败。
const char* host = mHost.starts_with('^') ? nullptr : mHost.c_str();
if (evaluate_domain_name(mNetContext, host)) { // 主机名合法性检查
// 讲解注释②：真正的解析入口 getaddrinfo.cpp:409 resolv_getaddrinfo——
// 内部先查 res_cache，未命中再发网络查询（UDP 明文或 DoT，见 11.3）。
rv = resolv_getaddrinfo(host, service, mHints.get(), &mNetContext, &result, &event);
} else {
rv = EAI_SYSTEM;
}
queryLimiter.finish(uid);
} else { ... }
doDns64Synthesis(&rv, &result, &event); // 讲解注释③：IPv6-only 网的 NAT64 合成：
const int32_t latencyUs = saturate_cast<int32_t>(s.timeTakenUs()); // A 记录补 64:ff9b 前缀
...
if (rv) {
success = !mClient->sendBinaryMsg(ResponseCode::DnsProxyOperationFailed, &rv, sizeof(rv));
} else {
success = !mClient->sendCode(ResponseCode::DnsProxyQueryResult);
addrinfo* ai = result; // 讲解注释④：应答是 addrinfo 链表，逐个序列化：
while (ai && success) { // sendBE32(1)+sendaddrinfo，最后 0 结尾——
success = sendBE32(mClient, 1) && sendaddrinfo(mClient, ai); // 与 bionic 的读取端严格对偶
ai = ai->ai_next;
}
success = success && sendBE32(mClient, 0);
}
...
reportDnsEvent(INetdEventListener::EVENT_GETADDRINFO, mNetContext, latencyUs, rv, ...,
mHost, ip_addrs, total_ip_addr_count); // 讲解注释⑤：指标上报——dumpsys
freeaddrinfo(result); // dnsresolver 的统计与 statsd 指标源
}
```

设计洞察 · 逐段精读：缓存的正反两面

res_cache（res_cache.cpp，per-netId 各一份）正面是 resolv_cache_lookup（:1230）：TTL 内直接返回命中，省 RTT 也省电；反面是 _resolv_cache_query_failed（:1103）：失败也缓存——连续超时的域名在退避窗口内直接快速失败，避免应用重试风暴把弱网打死。排障时"换个应用问同一个域名也秒失败"多半是失败缓存生效，等退避过期或切网即恢复。

11.3 DoT 校验：PrivateDnsConfiguration 与 DnsTlsTransport

Private DNS 模式由 CS 侧 DnsManager（:24-26 三态常量）经 netd 下发，resolver 侧的承接者是 PrivateDnsConfiguration.cpp——mode 记在 per-netId 的 map 里（:73-77：SPECIFIED→STRICT / opportunistic→OPPORTUNISTIC / 空→OFF），服务器变化触发校验线程。

cpp packages/modules/DnsResolver/PrivateDnsConfiguration.cpp:197-239 与 DnsTlsTransport.cpp:276-300（骨架+讲解注释）

```
std::thread validate_thread([this, identity, server, netId, isRevalidation] {
// 讲解注释①：校验在独立 detached 线程做——阻塞网络 IO（TCP+TLS 握手），
// 与 DnsProxyListener 的查询路径互不牵连；identity 是版本号，防止
// 校验期间配置又变了导致旧结果覆盖新配置。
const bool success = DnsTlsTransport::validate(server, server.validationMark()); // :220
LOG(WARNING) << "validateDnsTlsServer returned " << success << " for " ...;
...
});
validate_thread.detach();
// DnsTlsTransport.cpp:276——STRICT 模式的"准入考试"：
bool DnsTlsTransport::validate(const DnsTlsServer& server, uint32_t mark) {
// 讲解注释②（续）：mark 是 netId 的 socket 标记——保证这条 DoT 探测流量走正确的网络。
// 会话缓存（DnsTlsSessionCache）复用 TLS 握手，避免每次查询重付 1-RTT。
const std::vector<uint8_t> query = makeDnsQuery(); // 讲解注释②：出题：一条固定的
DnsTlsSocketFactory factory; // 预定义域名 A 查询
DnsTlsTransport transport(server, mark, &factory);
auto r = transport.query(netdutils::makeSlice(query)).get(); // 讲解注释③：真发一条
if (r.code != Response::success) { return false; } // DoT 查询：TCP 853+TLS 握手+查询
if (auto result = checkDnsResponse(r.response); !result.ok()) { return false; } // 验响应
// 讲解注释④：OPPORTUNISTIC 模式还比延迟：dot_probe_latency 与 UDP 探测延迟按
// latencyFactor/latencyOffsetMs 公式比较——TLS 太慢则不用，体现"尽力加密"语义；
// STRICT 不做此比较：只认可达性，不达即拒（第五章的"不回落"在 resolver 侧的落点）。
...
}
```

校验通过的服务器进入可用集（mark 为 netId 的 socket 标记，保证 DoT 流量走对网络）；此后正常查询里，若 mode=STRICT 则只对已验证服务器发 DoT（DnsTlsTransport::query，:151），全部失败即整体失败——**与明文 UDP 之间没有任何回落路径**。

> **设计洞察 · 设计洞察：为什么校验用"真查询"不用空握手**
>
> validate 完全可以只测"853 端口 TLS 能握手"，但它发了一条真实 DNS 查询并校验响应。原因：部分门户/防火墙允许 TCP 853 握手成功却在应用层拦截 DNS 载荷（策略设备只看端口不看内容是不完全的，反之亦然）；一条端到端真查询同时否决了两类假可用。这也是排障线索：Telnet 通 853 不代表 DoT 可用，必须用真实解析验证（见排障手册 10.3）。

教科书习题集（分级）

第一级：简答题（检验概念理解）

- **1. 本树 Connectivity 主代码在哪个目录？Vpn.java 为什么被称作"活化石"？ 参考答案要点**：packages/modules/Connectivity/（Mainline APEX，com.android.tethering）；frameworks/base/services/core/.../connectivity/Vpn.java:179 是该目录唯一未迁移的实体，与系统用户管理耦合过深。
- **2. NetworkMonitor 的四探针是哪四个？各自失败分别意味着什么？ 参考答案要点**：PROBE_DNS/HTTP/HTTPS/PRIVDNS（NM.java:29-33）；DNS 失败=域名不通，HTTP 302=强制门户，HTTPS 不通 HTTP 通=部分连通，PRIVDNS 失败=Private DNS 不可用。
- **3. linger 是什么？默认多久？它为 App 解决什么问题？ 参考答案要点**：切网时老网的宽限期，默认 30s（mLingerDelayMs）；吸收 Wi-Fi 闪断等毛刺，宽限期内老网仍可服务已绑定 socket，避免频繁断连重建。
- **4. TrafficStats 的 tag 与 uid 分别由谁核对？伪造 tag 能逃账吗？ 参考答案要点**：tag 高 16 位 App 自报自用，uid 部分由内核 socket 属主核对（bpf_traffic_account 先取 sock_uid 再查 cookie_tag_map）；伪造 tag 逃不掉 uid 账。
- **5. Private DNS 三态是什么？STRICT 配错主机名的后果？ 参考答案要点**：OFF（明文 53）/STRICT（DoT 853，失败不回落）/AUTO（先 TLS 失败回落，DnsManager.java:24-26）；STRICT 配错=整体解析失败，安全语义高于可用性。
- **6. VPN 的 protect() 防的是什么环？缺失会发生什么？ 参考答案要点**：防"加密通道自身流量再被路由进 tun"的递归环；缺失后 VPN 流量套娃进自身隧道，连接死循环、网络全断。
- **7. eBPF 计账比 iptables 计数好在哪？ 参考答案要点**：map 查询 O(1)、可按 uid/tag/counterSet/iface 任意维度聚合、写入侧无需频繁改规则；iptables 计数 O(rules) 且动态改规则代价高。
- **8. registerNetworkAgent 返回时 NAI 已注册完成吗？中间还有什么阶段？ 参考答案要点**：没有——返回的是"准生证"（netId 已分配）；收尾在 handleRegisterNetworkAgent(:7345)：等 NetworkStack 进程回传 NetworkMonitor 后 mixInCapabilities、入 mNetworkAgentInfos 与 mNetworkForNetId 表。
- **9. makeDefault 除了改内核默认路由还联动哪些子系统？ 参考答案要点**：VPN 底层能力传播、per-app UID 规则（makeDefaultForApps）、LingerMonitor 宽限、代理（ProxyTracker）、TCP 缓冲（updateTcpBufferSizes）、记账 iface 通知（CS.java:8539-8602）。
- **10. PARTIAL_CONNECTIVITY 的判定条件是什么？用户接受后状态机怎么走？ 参考答案要点**：HTTPS 探针失败而 HTTP 探针成功；用户接受→maybeDisableHttpsProbing(true) 关 HTTPS 探针、报告 VALID+PARTIAL、转 EvaluatingPrivateDnsState 收尾。

第二级：分析题（检验机制理解）

- **11. 追踪 registerNetworkAgent→handleRegisterNetworkAgent 全链，标注每一次跨进程跳。 参考答案要点**：①agent 进程→CS（binder 注册）；②CS→NetworkStack 进程（makeNetworkMonitor 创建 NM）；③NetworkStack→CS（INetworkMonitor 代理回传+回调）

；收尾入表并 start()，事件由 NetworkAgent 暂存重放保证不乱序。

- **12. Wi-Fi 信号渐弱到热点关闭：从 RSSI 降分画出完整时序（含 rematch、linger、teardown）。 参考答案要点**：WifiScoreReport 降分→CS scoreChanged→rematch（computeNetworkReassignment :8802）→蜂窝为 bestNetwork→processDefaultNetworkChanges→makeDefault→networkSetDefault+老网 noteLingerDefaultNetwork→30s 后 handleLingerComplete(:8499)→unneeded(:4601)→teardownUnneededNetwork(:8486)。
- **13. 若 NetworkStack 进程被杀，已验证网络会怎样？默认网有何影响？ 参考答案要点**：NM 实例随进程消失，CS 经 binder 死亡回调重建 makeNetworkMonitor 并重新验证；期间验证结论维持旧值，默认网不立即易主；门户网可能"假在线"，极端情况反复 rematch。
- **14. 对比 per-app 默认网（makeDefaultForApps）与 per-app VPN 的分流量机制。 参考答案要点**：前者网络级——netd 的 UID 范围规则把指定 UID 路由到指定 netId（networkAddUidRangesParcel）；后者隧道级——Vpn UID 白名单+全量路由劫持+protect；前者无加密无额外跳，后者多一次用户态转发与加解密。
- **15. stats_map_A/B 双 map 设计要解决什么竞争？如何切换？ 参考答案要点**：解决"读侧导出期间写侧继续累加导致快照不一致"；configuration_map 存活 map 指针，netd 切换后旧 map 冻结只读、从容导出后清零轮换；无锁，代价仅一次指针翻转（netd.c:78-79）。
- **16. 解释一条 DNS 查询从 bionic fprintf 到 res_cache 命中/未命中的两条路径。 参考答案要点**：android_getaddrinfo_proxy(getaddrinfo.c:406) 写文本行协议到 /dev/socket/dnsproxyd（NetdClient.cpp:311）→DnsProxyListener 解析命令 spawn Handler(:729)→queryLimiter→resolv_getaddrinfo(getaddrinfo.cpp:409)：命中 resolv_cache_lookup(res_cache.cpp:1230) 直接返回；未命中发网络查询并回填，失败记快速失败缓存。
- **17. 分析 STRICT 模式下校验线程与查询路径的关系：校验没通过时查询去哪了？ 参考答案要点**：PrivateDnsConfiguration 的 validate 线程(:197)独立于查询路径，通过后服务器才进可用集；未通过期间 STRICT 查询无可用 DoT 服务器→整体失败（不回落明文）；AUTO 模式失败后由 dispatcher 转明文 UDP。

第三级：设计题

- **18. 若把验证主探针从 HTTP generate_204 换成 QUIC 探测，需要动哪些面？如何向后兼容？ 参考答案要点**：动 NetworkStack（新探针类型与判定）、capport API 协商、CS 结果位语义、指标枚举；兼容：保留 HTTP/HTTPS 并行做门户检测与回退（QUIC 表达不了 302），Feature flag 分批灰度——NetworkStack 可独立升级正是为此预留。
- **19. 设计"切网零丢包"方案：结合 linger、TCP keepalive、QUIC connection migration 各自的分工。 参考答案要点**：linger 保老网 30s 可用；TcpKeepaliveController 把空闲连接移植到新网（keepalive 卸载到网卡）；QUIC 用 connection ID 迁移换路无感；应用侧 registerNetworkCallback+bindProcessToNetwork；分层组合答案优于任何单点方案。
- **20. 设计多设备热点流量限额功能：说明需要哪些 map、轮询与告警链。 参考答案要点**：per-upstream TetherStatsValue 已有（offload.c tc 程序写入）；

BpfCoordinator.updateQuotaAndStatsFromSnapshot(:1869) 差分供告警（updateAlertQuota→notifyAlertReached）；按客户端细分需在 offload 程序加 per-client map；超限动作走 netd 防火墙规则；5s 轮询精度权衡要写进设计。

术语表

术语	解释
cookie_tag_map	eBPF 计账 map：socket cookie→uid+tag，tagSocket 写入、bpf_traffic_account 读取（netd.c:75）
counterSet	前后台计数集：per-uid 的前台/后台标记，进 StatsKey 参与聚合（netd.c:76）
dnsproxyd	DNS 代理 unix socket：bionic 经它把 getaddrinfo 递进 resolver（NetdClient.cpp:311）
DoT	DNS over TLS：系统 Private DNS 的加密传输（TCP 853），仅 STRICT/AUTO 模式使用
eBPF	扩展伯克利包过滤器：内核虚拟机程序，本册承担计账、防火墙与热点转发卸载
FullScore	网络全分值：代理分+能力策略位的统一计分对象，Ranker 择优的依据
linger	切网宽限期：老默认网在 30s 内保持可用，吸收切换毛刺（LingerMonitor）
netd	网络守护进程：CS 的 native 执行体（system/netd/server/main.cpp:111），DNS 监听与规则落地
netId	网络标识号：CS 每网络分配的编号，对应内核 per-network 路由表
NetworkAgent	网络代理：Wi-Fi/蜂窝等供给侧在各自进程的状态机，经 AIDL 向 CS 注册
NetworkAgentInfo	NAI：一个活网络的运行时视图（agent+FullScore+linkProperties+验证状态）
NetworkFactory	网络工厂：供给侧声明"我有什么"的评估入口
NetworkMonitor	网络监视器：NetworkStack 进程内 per-网络验证状态机（NM.java:225）
NetworkRanker	网络排序器：在满足请求能力的网络中选 FullScore 最高者
PARTIAL_CONNECTIVITY	部分连通：HTTPS 不通 HTTP 可用的验证结论，用户可选"仍使用"
Private DNS	私有 DNS：系统级加密解析，三态 OFF/STRICT/AUTO（DnsManager.java:24-26）

术语	解释
probe	验证探针：四类（DNS/HTTP/HTTPS/PRIVDNS），子线程执行、消息回灌状态机
强制门户	captive portal：需网页登录的网络，HTTP 探针 302 触发登录通知
resolv 缓存	per-netId DNS 缓存：TTL 命中省 RTT，失败缓存防重试风暴（res_cache.cpp:1230）
tagSocket	套接字打标：新 socket 创建时按 cookie 写 uid_tag 映射，记账第一环
Tethering offload	热点转发卸载：包走硬件直转不上 CPU，per-iface 计数由 BpfCoordinator 轮询
TUN 设备	虚拟网卡：VPN 的用户态出口（TUNSETIFF 建立），read/write 即收发包
uid_owner_map	UID 规则 map：后台禁网/省流等放行与禁止判决（netd.c:81）

排障手册：网络栈问题定位

定位三步法：1. dumpsys connectivity 定位状态（哪个网、验证态、分数、linger）；2. logcat 查日志（NetworkMonitor/ConnectivityService/resolv 标签）；3. 按症状深入对应章节与下面的场景清单。

10.1 网络切换实战：Wi-Fi ↔ 蜂窝的无缝切换

text 网络切换的完整时序（读码归纳）

```
t0: Wi-Fi 信号减弱（RSSI < 阈值）→ WifiScoreReport 降分
t1: CS 收到 scoreChanged → rematch → 蜂窝分 > Wi-Fi 分
t2: 蜂窝成为 defaultNet → 广播 NETWORK_CAPABILITY_CHANGED
t3: Wi-Fi 进入 linger（默认 30s）→ 老连接继续
t4: 新连接走蜂窝 → App 无感（TCP keepalive 保持）
t5: 30s 后 Wi-Fi 真断 → linger 结束 → teardown
// 讲解注释①：App 侧怎么感知?
// ConnectivityManager.registerNetworkCallback(NET_CAPABILITY_INTERNET)
// → onCapabilitiesChanged（新网的 capability）
// → onLosing（老网即将断，maxMsToLive 可查剩余宽限）
// → onLost（老网已断）
// 讲解注释②：双栈应用（Wi-Fi+蜂窝同时活跃）→ bindProcessToNetwork
```

10.2 Wi-Fi 已连接但提示"无法访问互联网"

症状：状态栏 Wi-Fi 正常但部分应用报无网，或通知"已连接但无法访问互联网"；浏览器能开登录页。

定位命令：`adb shell dumpsys connectivity` 看该网络 NetworkAgentInfo 的 VALIDATED 字段与 partialConnectivity 标记；`adb shell "logcat -d -s NetworkMonitor"` 看探针结论（isSuccessful/isPortal/RedirectUrl）；`adb shell curl -v http://connectivitycheck.gstatic.com/generate_204` 手动复现探针（看返回 204 还是 302/200）。

根因：三类判例——探针被门户 302 劫持（进 CaptivePortalState，第九章）；HTTPS 域名被丢包/劫持而 HTTP 正常（PARTIAL）；企业防火墙拦 generate_204 域名（INVALID 循环退避）。

修复：门户网完成登录（或企业放行 connectivitycheck 域名）；长期 PARTIAL 检查 HTTPS 出口链路；确属误判可用 `adb shell cmd netpolicy` 侧命令与"仍使用此网络"按钮接受部分连通。

10.3 DNS 解析慢或部分域名失败

症状：网页首包动辄秒级、偶发 UnknownHostException；切换网络后加重；特定域名稳定失败。

定位命令：`adb shell dumpsys dnsresolver` 看服务器列表/缓存命中/解析事件统计；`adb shell settings get global private_dns_mode` 与 `adb shell settings get global private_dns_hostname` 核对模式；`adb shell "logcat -d -s resolv"` 看 DnsTls 校验与查询失败原因。

根因：STRICT 模式下 DoT 服务器 853 端口不可达（校验线程反复失败，第十一章 11.3）→ 整体解析失败不回落；AUTO 模式在"TLS 可达但慢"的网络里反复回落抖动；失败缓存（res_cache 快速失败）让重试看起来"更坏"。

修复：STRICT 换可达服务商或改 AUTO；确认 853 出口未被封；理解失败缓存语义——排障时换 App 或等退避过期再测，避免把缓存失败当实时故障。

10.4 VPN 无法建立（establish 返回 null 或抛异常）

症状：VpnService.Builder.establish() 返回 null，或系统弹不出同意框；企业 VPN 应用反复重连失败。

定位命令：`adb shell "dumpsys connectivity"` 搜 mVpn 节（接口名/owner UID/always-on 状态）；`adb shell "logcat -d -s Vpn"` 看 establish 抛的具体 SecurityException；`adb shell ip addr show tun0` 确认 tun 是否真的建了；`adb shell settings get global always_on_vpn_app` 查 lockdown 占用者。

根因：五类——prepare 未完成（mOwnerUID 与调用者不符，Vpn.java:1505）；目标 Service 未声明 BIND_VPN_SERVICE 或无法被 resolveService 解析（:1522-1527）；受限用户被拒（:2318）；lockdown 被另一个 VPN 占用；旧 tun 未释放导致 jniGetName/设地址失败。

修复：先调 VpnService.prepare() 拿用户同意；核对 manifest 的 VpnService 声明与权限；lockdown 冲突先关闭原 VPN 的 always-on；失败重试前确保旧 establish 的 fd 已关闭（VPN 应用通常忘了上一轮的 ParcelFileDescriptor）。

10.5 流量统计不准（Settings 流量页与实际不符）

症状：开热点后主卡流量没涨；某应用流量显示翻倍；切应用后台流量归属错。

定位命令：`adb shell dumpsys netstats detail` 对比 per-uid 与 per-iface 口径；`adb shell dumpsys connectivity tether` 看 offload 状态与 BpfCoordinator 轮询；`adb shell dumpsys netd` 看计账 map 配置（当前写的是 A 还是 B）。

根因：热点 offload 期间流量记在 TetherStatsValue（per-iface），BpfCoordinator 每 5s 才差分上报一次（第十章 10.3）——"不实时"是设计；翻倍常见于 464xlat 场景下 clat 特判未生效的旧内核；归属错多为应用 setThreadStatsTag 后未 clearThreadStatsTag，tag 泄漏到线程后续 socket。

修复：热点流量以 tether 统计为准（有 5s 粒度延迟）；老设备升级内核启用 v4 offload 门控（5.8+）；应用侧打 tag 用 try/finally 保证清理；跨网切换瞬间统计误差属周期采样正常现象，勿当 bug 追。

10.6 默认网切换后代理失效

症状：Wi-Fi（带 PAC/静态代理）切蜂窝后应用仍试图走旧代理，或反之；企业内网代理在切网后断流。

定位命令：`adb shell dumpsys connectivity` 看各网络 linkProperties 的 httpProxy 字段；`adb shell "logcat -d -s ProxyTracker"`；`adb shell dumpsys connectivity "proxy"`（部分版本支持代理子命令）。

根因：代理是 LinkProperties 的一部分，随 makeDefault 的 handleApplyDefaultProxy 联动（第八章 8.2）；App 缓存了旧 ProxyInfo（未监听 onCapabilitiesChanged/PROXY 变更回调）是最常见根因，而非系统未切换。

修复：应用改用 ProxySelector/系统回调获取实时代理；系统侧核对 makeDefault 联动链是否被定制 ROM 改动破坏（对照 CS.java:8563-8568 原生行为）。

网络栈核心速查表

机制	一句话	源码锚点	所在册
Mainline 路径	Connectivity→packages/modules/Connectivity	CS/ConnectivityService.java:314	本册
NetworkMonitor	NetworkStack 独立模块（StateMachine）	NS/NetworkMonitor.java:225	本册
四探针	DNS/HTTP/HTTPS/PRIVDNS	NM:29-33	本册
评分择优	NetworkRanker+FullScore	CS/NetworkRanker	本册

机制	一句话	源码锚点	所在册
linger	切网宽限期（默认 30s）	CS/LingerMonitor	本册
eBPF 记账	tagSocket→cookie map→per-uid 计数	BpfNetMaps.java:37	本册
Tethering offload	BpfCoordinator 硬件转发不上 CPU	Tethering/IpServer	本册
Private DNS	OFF/STRICT(DoT 853)/AUTO（先 TLS 回落）	DnsManager:24-26	本册
VPN	tun0+路由劫持+protect 防环	Vpn.java:1504	本册
多网络并发	ProfileNetworkPreferenceList 按 profile 分流	CS	本册